Tema 001.

APD1. Implementați în C# folosind Parallel.For pe CPU înmulțirea a două matrice dense. Împărțiți matricea rezultat în blocuri dreptunghiulare. Fiecare iterație a buclei Parallel.For calculează toate elementele unui bloc. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, testați câte trei dimensiuni diferite ale blocului, la aceeași valoare a MaxDegreeOfParallelism. Includeți o configurație în care un bloc de la margine are mai puține linii sau coloane. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă de partiționare care arată ce elemente din cele două matrice de intrare sunt folosite pentru un bloc complet și pentru unul de la margine. Scrieți pseudocodul calculului unui bloc, cu limitele liniilor și coloanelor și suma produselor pentru fiecare element. Realizați o schemă logică a programului, de la pregătirea datelor până la verificarea rezultatului după terminarea buclei Parallel.For.

Termen predare: săptămâna 9

Tema 002.

APD1. Implementați în C# pe CPU înmulțirea a două matrice dense folosind threaduri create explicit. Împărțiți liniile matricei rezultat în intervale consecutive cât mai egale ca lungime. Atribuiți fiecărui thread un interval înainte de pornire și folosiți Thread.Join înainte de a citi rezultatul. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, testați trei valori ale numărului de threaduri. Includeți un caz în care numărul de linii nu este multiplu al numărului de threaduri. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri.

IS. Realizați o schemă de partiționare pe linii pentru șapte linii și trei threaduri. Scrieți pseudocodul calculului limitelor intervalului atribuit unui thread și al prelucrării lui. Realizați o diagramă de timp care arată pornirea threadurilor, așteptarea lor cu Thread.Join și momentul citirii rezultatului.

Termen predare: săptămâna 9

Tema 003.

APD1. Implementați în C# pe CPU înmulțirea a două matrice dense folosind threaduri create explicit. Împărțiți coloanele matricei rezultat în blocuri de coloane consecutive. Atribuiți blocurile ciclic înainte de pornirea threadurilor. Fiecare thread calculează toate liniile din coloanele care i-au fost atribuite. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, testați trei lățimi ale blocului la același număr de threaduri. Includeți o lățime pentru care ultimul bloc are mai puține coloane. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri.

IS. Realizați o schemă de repartizare ciclică a blocurilor pentru trei threaduri, cu un ultim bloc incomplet. Scrieți pseudocodul calculului indicilor blocurilor și coloanelor atribuite unui thread. Realizați o schemă logică a parcurgerii blocurilor de către un thread, de la primul bloc atribuit până la terminare.

Termen predare: săptămâna 9

Tema 004.

APD1. Implementați în C# pe CPU înmulțirea a două matrice dense folosind threaduri create explicit. Împărțiți liniile matricei rezultat în blocuri consecutive. Fiecare thread preia indicele următorului bloc disponibil folosind Interlocked.Increment pe un contor comun. După ce calculează blocul, threadul solicită alt bloc până la epuizarea lor. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, testați trei valori ale numărului de linii dintr-un bloc. Păstrați același număr de threaduri și raportați și numărul de blocuri calculate de fiecare thread, într-o rulare separată. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri.

IS. Realizați o schemă de partiționare pe blocuri de linii și indicați corespondența dintre indicele blocului și liniile calculate. Scrieți pseudocodul buclei unui thread, inclusiv condiția de oprire. Realizați o diagramă de secvență în care două threaduri solicită blocuri când mai este disponibil un singur bloc.

Termen predare: săptămâna 9

Tema 005.

APD1. Implementați în C# pe CPU înmulțirea a două matrice dense folosind Parallel.ForEach și Partitioner.Create. Partiționați indicii liniilor matricei rezultat în intervale consecutive. Fiecare iterație primește un interval și calculează toate elementele de pe liniile respective. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, comparați trei lungimi ale intervalului la aceeași valoare a MaxDegreeOfParallelism. Includeți un caz cu mai multe intervale decât limita de paralelism și raportați numărul total de intervale pentru fiecare configurație. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă care arată cum sunt grupate liniile în intervale, inclusiv ultimul interval incomplet. Scrieți pseudocodul corpului buclei Parallel.ForEach, folosind limita superioară exclusivă. Realizați o diagramă de secvență care arată preluarea mai multor intervale de către threadurile participante și revenirea din Parallel.ForEach.

Termen predare: săptămâna 9

Tema 006.

APD1. Implementați în C# pe CPU înmulțirea a două matrice dense folosind taskuri. Porniți de la intervalul care cuprinde toate liniile matricei rezultat. Dacă intervalul depășește un prag, împărțiți-l în două și executați un subinterval într-un task, iar pe celălalt în apelul curent. Așteptați taskul înainte de a reveni din apel. Calculați secvențial intervalele care ating pragul sau sunt mai mici. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, testați trei valori ale pragului. Pentru fiecare configurație, determinați numărul de taskuri create într-o rulare separată. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp.

IS. Desenați arborele de descompunere pentru un număr impar de linii și indicați intervalele procesate secvențial. Scrieți pseudocodul apelului recursiv, cu pragul și punctul de așteptare. Realizați o diagramă de timp care arată execuția celor două subintervale și revenirea din apelul care le-a creat.

Termen predare: săptămâna 9

Tema 007.

APD1. Implementați în C# pe CPU înmulțirea a două matrice dense folosind Parallel.For. Împărțiți dimensiunea comună a matricelor în intervale consecutive. Pentru fiecare interval, calculați o matrice parțială, care conține pentru fiecare element al rezultatului suma produselor corespunzătoare acelui interval. După terminarea buclei, însumați secvențial matricele parțiale. Folosiți date întregi și un tip numeric suficient pentru rezultate și sumele intermediare. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, testați trei numere de intervale la aceeași valoare a MaxDegreeOfParallelism. Raportați și memoria ocupată de elementele matricelor parțiale. Includeți însumarea în timpul măsurat. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă de partiționare care asociază coloanele primei matrice cu liniile celei de-a doua pentru fiecare interval. Scrieți pseudocodul calculului unei matrice parțiale și al însumării. Realizați o diagramă de flux al datelor care arată cum contribuie toate matricele parțiale la același element din matricea rezultat.

Termen predare: săptămâna 9

Tema 008.

APD1. Implementați în C# pe CPU înmulțirea a două matrice dense după modelul producător–consumator. Un thread producător introduce într-o coadă cu capacitate limitată coordonatele blocurilor matricei rezultat. Folosiți BlockingCollection pentru coadă. Fiecare thread consumator preia coordonatele unui bloc și calculează toate elementele acelui bloc. Producătorul semnalează încheierea adăugării, iar consumatorii continuă până la golirea cozii. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, comparați trei capacități ale cozii. Păstrați dimensiunea blocului și numărul de consumatori constante. Includeți în timpul paralel distribuirea blocurilor prin coadă. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul total de threaduri producătoare și consumatoare.

IS. Realizați o schemă care leagă coordonatele din coadă de blocurile matricei rezultat. Scrieți pseudocodul producătorului și al unui consumator, inclusiv condițiile de terminare. Realizați o diagramă de stări a consumatorului care arată așteptarea când coada este temporar goală și terminarea după golirea cozii închise.

Termen predare: săptămâna 9

Tema 009.

APD1. Implementați în C# pe CPU înmulțirea a două matrice dense cu un grup fix de threaduri. În prima etapă, distribuiți liniile matricei a doua între threaduri și construiți transpusa. Folosiți o barieră înainte de etapa următoare. În a doua etapă, distribuiți liniile matricei rezultat între aceleași threaduri și folosiți transpusa pentru a calcula produsele

scalare pe linii. Folosiți aceleași două etape și în referința secvențială. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, testați trei valori ale numărului de threaduri. Includeți atât transpunerea, cât și înmulțirea în timpul de execuție. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri.

IS. Realizați o schemă care arată cum se folosesc o linie din prima matrice și o linie din matricea transpusă pentru a calcula un element al rezultatului. Scrieți pseudocodul celor două etape executate de un thread. Realizați o diagramă de timp în care threadurile termină transpunerea în momente diferite și marcați când poate începe înmulțirea.

Termen predare: săptămâna 9

Tema 010.

APD1. Implementați în C# pe CPU înmulțirea a două matrice dense folosind Parallel.Invoke. Împărțiți matricea rezultat în patru blocuri, separând atât liniile, cât și coloanele în câte două intervale. Transmiteți către Parallel.Invoke câte o acțiune pentru fiecare bloc. Fiecare acțiune calculează toate elementele blocului corespunzător. Comparați două moduri de alegere a limitelor: la jumătate pe ambele dimensiuni și la aproximativ un sfert pe ambele dimensiuni. Verificați rezultatul față de înmulțirea secvențială. Pentru două perechi de matrice, testați ambele împărțiri la aceeași valoare a MaxDegreeOfParallelism, fixată la patru. Includeți o pereche cu număr impar de linii sau coloane în matricea rezultat. Măsurați timpul de execuție pentru înmulțirea completă, cu matricele de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare variantă, repetați fiecare măsurare de cel puțin cinci ori și raportați media timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel pentru configurația măsurată. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă de partiționare în patru blocuri pentru fiecare dintre cele două împărțiri. Scrieți pseudocodul generării celor patru acțiuni din indicii care delimitează blocurile. Realizați o diagramă de dependențe cu cele patru acțiuni și punctul în care matricea rezultat devine disponibilă integral.

Termen predare: săptămâna 9

Tema 011.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți. Fiecare contor indică numărul de elemente din clasa sa. Împărțiți vectorul în p intervale consecutive, câte unul pentru fiecare thread. În prima variantă, actualizați o histogramă comună cu Interlocked.Increment. În a doua variantă, fiecare thread calculează o histogramă locală, iar threadul principal însumează histogramele locale după terminarea tuturor threadurilor. Comparați cele două variante pentru valori distribuite uniform în domeniu și pentru valori concentrate în două clase. Păstrați aceeași lungime a vectorului, aceleași valori M și K și același număr de threaduri. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație, repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri de calcul.

IS. Realizați o schemă de partiționare care leagă intervalele vectorului de histograma comună și, separat, de histogramele locale. Scrieți pseudocodul ambelor variante, inclusiv însumarea finală. Realizați o diagramă de secvență în care două threaduri actualizează contorul aceleiași clase din histograma comună.

Termen predare: săptămâna 9

Tema 012.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți. Fiecare contor indică numărul de elemente din clasa sa. Împărțiți vectorul în blocuri de poziții consecutive. Creați p threaduri care preiau următorul bloc disponibil prin incrementarea atomică a unui indice comun. Fiecare thread folosește aceeași histogramă locală pentru toate blocurile preluate. Însumați histogramele locale după terminarea threadurilor. Pentru două lungimi ale vectorului, testați trei dimensiuni ale blocului la același p. Includeți un ultim bloc incomplet. Determinați într-o rulare separată câte blocuri a procesat fiecare thread. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație, repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri de calcul.

IS. Realizați o schemă care arată limitele blocurilor, inclusiv ale ultimului bloc. Scrieți pseudocodul preluării blocurilor și al actualizării histogramei locale. Realizați o diagramă de secvență în care două threaduri solicită un bloc când mai este disponibil unul singur.

Termen predare: săptămâna 9

Tema 013.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți. Fiecare contor indică numărul de elemente din clasa sa. Creați p threaduri și atribuiți fiecăruia un interval de poziții din vector. Fiecare thread calculează o histogramă locală. După o barieră, fiecare thread însumează contoarele histogramelor locale pentru un interval disjunct de clase din histograma finală. Folosiți aceleași threaduri în ambele etape. Pentru două valori ale lui K și un M fix, variați p. Păstrați același vector de intrare. Includeți o configurație în care K nu este multiplu de p. Raportați și timpul etapei de însumare, măsurat separat de rulările folosite pentru timpul total. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație, repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri de calcul.

IS. Realizați o schemă care arată partiționarea pozițiilor în prima etapă și partiționarea claselor în a doua. Scrieți pseudocodul celor două etape executate de un thread. Realizați o diagramă de timp cu trei threaduri care ajung la barieră în momente diferite.

Termen predare: săptămâna 9

Tema 014.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți. Fiecare contor indică numărul de elemente din clasa sa. Folosiți Parallel.For cu inițializare locală și o acțiune finală pentru fiecare stare locală. În corpul buclei, actualizați histograma asociată stării locale. În acțiunea finală, combinați histograma locală cu histograma comună. Comparați protejarea întregii însumări cu un singur lock și însumarea prin Interlocked.Add pentru fiecare clasă cu un contor local nenul. Pentru două valori ale lui K la același M, comparați mecanismele de însumare la trei valori ale MaxDegreeOfParallelism. Păstrați același vector și numărați stările locale create într-o rulare separată. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație,

repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o diagramă de flux al datelor de la inițializarea stării locale până la actualizarea histogramei comune. Scrieți pseudocodul celor două acțiuni finale. Realizați o diagramă de secvență cu două acțiuni finale care încearcă să actualizeze aceeași clasă, în ambele variante.

Termen predare: săptămâna 9

Tema 015.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți. Fiecare contor indică numărul de elemente din clasa sa. Folosiți modelul producător–consumator. Un thread producător introduce într-o BlockingCollection cu capacitate limitată intervalele de poziții din vector. Threadurile consumatoare preiau intervalele și actualizează fiecare propria histogramă locală. După golirea cozii și terminarea consumatorilor, însumați histogramele locale. Testați trei capacități ale cozii pentru același vector, aceeași dimensiune a blocului și același număr de consumatori. Includeți în timpul paralel pornirea threadurilor, transferul intervalelor prin coadă și însumarea. Calculați și numărul de elemente procesate pe secundă. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație, repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul total de threaduri producătoare și consumatoare.

IS. Realizați o schemă a fluxului de date prin coadă și a însumării histogramelor locale. Scrieți pseudocodul producătorului și al unui consumator. Realizați o diagramă de stări a consumatorului care distinge așteptarea la o coadă temporar goală de terminarea după semnalarea finalului și golirea cozii.

Termen predare: săptămâna 9

Tema 016.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți. Fiecare contor indică numărul de elemente din clasa sa. Folosiți taskuri pentru a calcula câte o histogramă locală pentru fiecare bloc de poziții consecutive. Însumați histogramele în perechi, pe niveluri, folosind câte un task pentru fiecare pereche. Așteptați terminarea unui nivel înainte de a începe nivelul următor. Dacă rămâne o histogramă fără pereche, transferați-o la nivelul următor. Pentru două lungimi ale vectorului, testați trei numere de blocuri, dintre care cel puțin unul impar. Păstrați M și K constante. Includeți toate nivelurile de însumare în timpul măsurat și raportați numărul de taskuri folosite la însumare. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație, repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Desenați arborele de însumare pentru cinci blocuri. Scrieți pseudocodul unui nivel, inclusiv tratarea histogramei fără pereche. Realizați o diagramă de timp care arată ce însumări pot rula simultan și când începe nivelul următor.

Termen predare: săptămâna 9

Tema 017.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți.

Fiecare contor indică numărul de elemente din clasa sa. Împărțiți vectorul în p intervale de poziții, câte unul pentru fiecare thread. Folosiți o histogramă comună și împărțiți clasele în grupe consecutive. Asociați fiecărei grupe un obiect lock. Pentru fiecare element, threadul calculează clasa, intră în secțiunea critică a grupei corespunzătoare și incrementează contorul clasei. Comparați trei numere de grupe, inclusiv o singură grupă, pentru un vector cu valori uniforme și unul cu valori concentrate în câteva clase. Păstrați lungimea vectorului, M, K și p constante. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație, repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri de calcul.

IS. Realizați o schemă care arată corespondența dintre clase, grupe și obiectele lock. Scrieți pseudocodul actualizării unui contor. Realizați o diagramă de secvență pentru două threaduri care actualizează clase diferite din aceeași grupă, apoi pentru clase din grupe diferite.

Termen predare: săptămâna 9

Tema 018.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți. Fiecare contor indică numărul de elemente din clasa sa. În prima etapă, folosiți Parallel.For pentru a înlocui fiecare valoare dintr-un vector auxiliar cu indicele clasei sale, păstrând vectorul de intrare. În a doua etapă, calculați histograme locale pe blocuri din vectorul auxiliar și însumați-le. Comparați cu o variantă care folosește aceleași blocuri, dar calculează indicele clasei direct la actualizarea histogramei locale. Pentru două lungimi ale vectorului și trei dimensiuni ale blocului, comparați variantele la aceeași valoare a MaxDegreeOfParallelism. Includeți crearea și completarea vectorului auxiliar în timpul variantei cu două etape. Raportați memoria ocupată de elementele vectorului auxiliar. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație, repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o diagramă de flux al datelor pentru varianta cu vector auxiliar. Scrieți pseudocodul actualizării histogramei locale în fiecare variantă. Realizați o schemă logică a celor două etape care arată când este complet vectorul auxiliar și când începe numărarea.

Termen predare: săptămâna 9

Tema 019.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți. Fiecare contor indică numărul de elemente din clasa sa. Folosiți Parallel.For pentru a procesa blocuri de poziții consecutive. Comparați o variantă în care fiecare bloc are un vector local de K contoare cu o variantă în care fiecare bloc are un Dictionary<int, long> care stochează numai clasele întâlnite. În ambele cazuri, combinați rezultatele locale secvențial într-un vector final de K contoare. Pentru aceeași lungime a vectorului, aceleași M și K și aceeași partiționare, comparați date care ating puține clase cu date care ating multe clase în fiecare bloc. Folosiți același MaxDegreeOfParallelism. Raportați și numărul total de clase distincte stocate în dicționarele locale, însumat peste blocuri. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație, repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă a reprezentărilor locale pentru un bloc care conține valori din doar trei clase. Scrieți pseudocodul actualizării dicționarului local și al însumării lui în histograma finală. Realizați o diagramă de flux al datelor care arată cum sunt combinate două dicționare cu clase comune și clase distincte.

Termen predare: săptămâna 9

Tema 020.

APD1. Se dă un vector de valori întregi din intervalul [0, M−1]. Implementați în C# pe CPU calculul histogramei cu K clase, unde 1 ≤ K ≤ M. Încadrați valoarea x în clasa cu indicele ⌊xK/M⌋. Calculați produsul xK cu aritmetică pe 64 de biți. Fiecare contor indică numărul de elemente din clasa sa. Calculați histograma pentru un vector format din blocuri consecutive, fiecare deja sortat crescător. Folosiți Parallel.For pentru a procesa câte un bloc pe iterație. În fiecare bloc, parcurgeți indicii claselor în ordine și adunați la contorul local lungimea fiecărui interval consecutiv de elemente din aceeași clasă. Combinați histogramele locale după terminarea buclei. Folosiți o referință secvențială care încadrează și numără fiecare element. Pentru două lungimi ale vectorului, folosiți două valori ale lui K și păstrați același M, aceeași dimensiune a blocului și același MaxDegreeOfParallelism. Cronometrați ambele variante cu aceleași blocuri deja sortate în memorie. Raportați numărul intervalelor consecutive de elemente din aceeași clasă, însumat peste blocuri. Verificați toate contoarele față de histograma secvențială. Măsurați timpul de execuție de la inițializarea structurilor de calcul până la obținerea histogramei finale, cu vectorul de intrare deja în memorie. Folosiți aceleași date pentru rulările secvențiale și paralele. După o rulare de încălzire pentru fiecare configurație, repetați măsurarea de cel puțin cinci ori și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă a unui bloc sortat care conține mai multe valori distincte din aceeași clasă și arătați limitele intervalelor de clasă. Scrieți pseudocodul detectării schimbării clasei, inclusiv actualizarea ultimului contor. Realizați o diagramă de flux al datelor care arată cum contribuie două blocuri la același contor final.

Termen predare: săptămâna 9

Tema 021.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Împărțiți recursiv vectorul în două subintervale. Dacă lungimea intervalului depășește un prag, sortați unul dintre subintervale într-un task și pe celălalt în apelul curent. Așteptați terminarea taskului înainte de interclasare. La prag sau sub el, aplicați Merge Sort secvențial. Interclasați secvențial în toate apelurile. Pentru două lungimi ale vectorului, testați trei praguri pe aceleași date. Numărați taskurile create într-o rulare separată. Folosiți Merge Sort secvențial cu interclasare pe perechi drept referință pentru calculul accelerării. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Scrieți pseudocodul apelului recursiv, cu verificarea pragului. Desenați arborele de descompunere pentru un vector de lungime impară și marcați apelurile executate secvențial. Realizați o diagramă de timp cu un task și apelul curent, indicând când poate începe interclasarea.

Termen predare: săptămâna 10

Tema 022.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Împărțiți vectorul în blocuri consecutive de lungime cel mult b. Sortați blocurile cu Parallel.For, folosind Merge Sort secvențial în fiecare bloc. Interclasați iterativ perechi de intervale adiacente, cu o buclă Parallel.For pentru fiecare nivel. Citiți dintr-un vector și scrieți într-un al doilea vector. La sfârșitul nivelului, inversați rolurile vectorilor. Copiați în vectorul destinație intervalul rămas fără pereche. Pentru două lungimi ale vectorului, testați trei valori ale lui b la același MaxDegreeOfParallelism.

Includeți un caz cu un număr impar de blocuri inițiale. Folosiți Merge Sort secvențial cu interclasare pe perechi drept referință pentru calculul accelerării. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă de partiționare pentru cinci blocuri inițiale și primele două niveluri. Scrieți pseudocodul buclei pe niveluri, inclusiv copierea intervalului fără pereche. Realizați o diagramă de timp care arată când se pot inversa rolurile celor doi vectori.

Termen predare: săptămâna 10

Tema 023.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Creați p threaduri pe care le folosiți până la terminarea sortării. Distribuiți static blocurile inițiale între threaduri și sortați fiecare bloc cu Merge Sort secvențial. La fiecare nivel, distribuiți static perechile de interclasat. Citiți dintr-un vector și scrieți în altul, alternând rolurile lor după fiecare nivel. Folosiți o barieră după sortarea inițială și după fiecare nivel. Threadurile fără perechi atribuite participă și ele la barieră. Copiați în destinație orice interval rămas fără pereche. Pentru două lungimi ale vectorului, variați p, păstrând dimensiunea blocului inițial constantă. Includeți un nivel la care numărul perechilor este mai mic decât p. Folosiți Merge Sort secvențial cu interclasare pe perechi drept referință pentru calculul accelerării. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri de calcul.

IS. Realizați o schemă de atribuire a blocurilor și perechilor la trei threaduri. Scrieți pseudocodul unui thread, inclusiv cazul în care nu are nicio pereche atribuită. Realizați o diagramă de timp a barierei pentru un nivel cu mai puține perechi decât threaduri.

Termen predare: săptămâna 10

Tema 024.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Împărțiți vectorul în blocuri consecutive și construiți un arbore ale cărui frunze sortează blocurile cu Merge Sort secvențial, iar fiecare nod intern interclasează două intervale adiacente. Reprezentați fiecare nod printr-un task. Porniți interclasarea unui nod imediat ce taskurile celor doi copii s-au terminat, folosind Task.WhenAll și await. Fiecare nod intern produce un vector propriu. Dacă un interval rămâne fără pereche la construirea arborelui, transferați-l la nivelul următor. Pentru două lungimi ale vectorului, testați trei numere de blocuri. Măsurați și o variantă de control cu aceleași blocuri, aceleași operații și aceleași alocări, dar care așteaptă toate taskurile unui nivel înainte de a porni nivelul următor. Folosiți Merge Sort secvențial cu interclasare pe perechi drept referință pentru calculul accelerării. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Desenați arborele de dependențe pentru șase blocuri. Scrieți pseudocodul taskului unui nod intern. Realizați o diagramă de timp în care o interclasare începe cât timp o operație din altă ramură de la nivelul precedent este încă în execuție.

Termen predare: săptămâna 10

Tema 025.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Sortați recursiv cele două jumătăți cu taskuri, folosind un prag fix pentru trecerea la sortarea secvențială. Pentru interclasările care depășesc un al doilea prag, împărțiți intervalul destinație în segmente consecutive. Pentru fiecare limită de segment, determinați prin căutare binară câte elemente contribuie din fiecare subinterval sortat. La egalitate, considerați mai întâi elementele din subintervalul stâng. Interclasați independent porțiunile corespunzătoare fiecărui segment, în taskuri. Sub pragul de interclasare, folosiți interclasarea secvențială. Pentru două lungimi ale vectorului, testați trei praguri de interclasare. Păstrați constante pragul pentru sortarea recursivă și numărul maxim de segmente per interclasare. Comparați și cu aceeași sortare recursivă care interclasează întotdeauna secvențial. Folosiți Merge Sort secvențial cu interclasare pe perechi drept referință pentru calculul accelerării. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă care leagă două limite ale unui segment destinație de pozițiile din cele două subintervale, inclusiv pentru valori egale. Scrieți pseudocodul căutării binare a unei limite. Realizați o diagramă de dependențe pentru ultima interclasare, indicând calculele care pot rula în paralel.

Termen predare: săptămâna 10

Tema 026.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Sortați secvențial cele două jumătăți ale vectorului cu Merge Sort. Paralelizați numai ultima interclasare, folosind Parallel.For. Pentru fiecare element din jumătatea stângă, calculați poziția finală ca suma dintre indicele său local și numărul elementelor strict mai mici din jumătatea dreaptă. Pentru fiecare element din jumătatea dreaptă, adăugați la indicele local numărul elementelor mai mici sau egale din jumătatea stângă. Determinați aceste numere prin căutare binară și scrieți direct în vectorul destinație. Pentru două lungimi ale vectorului și trei valori ale MaxDegreeOfParallelism, comparați vectori cu valori distincte și vectori cu multe duplicate. Măsurați timpul total și, separat, timpul ultimei interclasări. Folosiți drept control aceleași sortări ale jumătăților, urmate de interclasarea secvențială obișnuită. Folosiți Merge Sort secvențial cu interclasare pe perechi drept referință pentru calculul accelerării. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă de corespondență între pozițiile de intrare și pozițiile finale pentru două jumătăți cu valori duplicate. Scrieți pseudocodul celor două căutări binare. Realizați o schemă logică ce arată când sunt disponibile jumătățile sortate și când poate fi citit vectorul destinație.

Termen predare: săptămâna 10

Tema 027.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Sortați secvențial blocuri consecutive cu Merge Sort, apoi interclasați perechile pe niveluri. La fiecare nivel, un thread producător introduce în BlockingCollection, cu capacitate limitată, descrierile perechilor. Threadurile consumatoare interclasează perechile și scriu în intervale disjuncte din vectorul destinație. Producătorul semnalează finalul listei. Așteptați terminarea consumatorilor înainte de a trece la nivelul următor. Copiați intervalul fără pereche în destinație înainte de a schimba rolurile vectorilor. Testați trei capacități ale cozii pentru două lungimi ale vectorului, păstrând dimensiunea blocului și

numărul consumatorilor constante. Includeți pornirea și terminarea threadurilor, operațiile pe coadă și copierea în timpul total. Folosiți Merge Sort secvențial cu interclasare pe perechi drept referință pentru calculul accelerării. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul total de threaduri producătoare și consumatoare.

IS. Realizați o schemă a fluxului perechilor de la producător la vectorul destinație. Scrieți pseudocodul execuției unui nivel. Realizați o diagramă de stări a consumatorului, cu așteptarea, interclasarea și terminarea după închiderea și golirea cozii.

Termen predare: săptămâna 10

Tema 028.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Identificați intervalele maximale deja ordonate crescător în sens larg din vector. Păstrați ordinea lor inițială și interclasați cu Parallel.For perechile de intervale adiacente de pe fiecare nivel. La fiecare nivel, construiți lista intervalelor obținute. Dacă un interval rămâne fără pereche, copiați-l în vectorul destinație și includeți-l în lista următoare. Comparați doi vectori de aceeași lungime, unul cu puține intervale inițiale lungi și unul cu multe intervale scurte. Testați trei valori ale MaxDegreeOfParallelism. Includeți identificarea intervalelor în măsurare și raportați numărul lor și numărul nivelurilor. Folosiți drept referință secvențială aceeași metodă cu interclasări executate pe rând. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă care arată limitele intervalelor inițiale într-un vector cu valori egale și scăderi. Scrieți pseudocodul identificării intervalelor și al construirii listei de la nivelul următor. Realizați o schemă logică a trecerii între niveluri până când rămâne un singur interval.

Termen predare: săptămâna 10

Tema 029.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Sortați blocurile inițiale cu Parallel.For și Merge Sort secvențial. La fiecare nivel, împărțiți bugetul de p unități de lucru între perechile de interclasat. Dacă sunt cel puțin p perechi, tratați fiecare pereche ca o singură unitate de lucru. Dacă sunt mai puține, alocați perechilor un număr de segmente proporțional cu lungimea lor, cu cel puțin un segment per pereche și cel mult p segmente în total. Pentru fiecare segment, identificați prin căutare binară porțiunile corespunzătoare din cele două intervale și interclasați-le în poziții disjuncte. La egalitate, selectați mai întâi elementele din intervalul stâng. Executați toate segmentele nivelului într-o singură buclă Parallel.For cu MaxDegreeOfParallelism egal cu p. Copiați separat intervalul fără pereche, dacă există. Pentru două lungimi ale vectorului, testați trei valori ale lui p. Păstrați aceeași dimensiune a blocurilor inițiale și comparați cu o variantă care interclasează fiecare pereche într-o singură iterație Parallel.For, la aceeași limită de paralelism. Măsurați separat și durata ultimului nivel. Folosiți Merge Sort secvențial cu interclasare pe perechi drept referință pentru calculul accelerării. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă de alocare a segmentelor pentru două perechi de lungimi diferite și p = 6. Scrieți pseudocodul distribuirii segmentelor și al construirii listei de lucru. Realizați o diagramă de timp care compară cele două variante la ultimul nivel.

Termen predare: săptămâna 10

Tema 030.

APD1. Implementați în C# pe CPU sortarea unui vector de numere întregi prin Merge Sort. Sortați independent blocurile inițiale cu Parallel.For, folosind Merge Sort secvențial. Împărțiți apoi blocurile sortate în grupe de câte patru blocuri adiacente, cu o ultimă grupă de cel mult patru blocuri. Folosiți Parallel.For pentru a procesa grupele unui nivel. În fiecare grupă, interclasați într-o singură trecere cu o coadă de priorități care păstrează cel mai mic element rămas din fiecare interval. Pentru valori egale, acordați prioritate intervalului aflat mai la stânga. Repetați gruparea până când obțineți un singur interval. Pentru două lungimi ale vectorului, comparați interclasarea pe grupe de patru cu interclasarea pe perechi. Păstrați dimensiunea blocurilor inițiale și MaxDegreeOfParallelism constante. Raportați numărul de niveluri și includeți un caz cu o ultimă grupă incompletă. Folosiți Merge Sort secvențial cu interclasare pe perechi drept referință pentru calculul accelerării. Verificați rezultatul față de referința secvențială, inclusiv pentru vectori cu valori duplicate. Măsurați timpul sortării complete, inclusiv alocarea vectorilor auxiliari și sincronizarea, după ce vectorul de intrare a fost încărcat în memorie. Pregătiți înainte de cronometrare copii identice ale vectorului nesortat. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă de grupare pentru nouă blocuri inițiale. Scrieți pseudocodul actualizării cozii de priorități după extragerea unui element. Realizați o diagramă de dependențe care arată când se pot interclasa rezultatele grupelor de pe nivelul precedent.

Termen predare: săptămâna 10

Tema 031.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Executați n faze, începând cu o fază pară. Folosiți Parallel.For în fiecare fază. Asociați fiecărei iterații o pereche, folosind indicele iterației și paritatea fazei. Începeți faza următoare după revenirea din buclă. Pentru două lungimi ale vectorului, testați trei valori ale MaxDegreeOfParallelism. Includeți o lungime impară. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă a perechilor din două faze consecutive. Scrieți pseudocodul calculului indicilor unei perechi. Realizați o diagramă de timp care arată când poate porni următoarea buclă Parallel.For.

Termen predare: săptămâna 10

Tema 032.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Executați n faze, începând cu o fază pară. Creați p threaduri la începutul sortării și reutilizați-le în toate fazele. Împărțiți indicii perechilor din fiecare fază în intervale consecutive atribuite static threadurilor. Folosiți o barieră după fiecare fază, inclusiv pentru threadurile fără perechi atribuite. Pentru două lungimi ale vectorului, variați p. Includeți un caz în care numărul perechilor nu este multiplu de p. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o

rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri de calcul.

IS. Realizați o schemă de partiționare pentru trei threaduri. Scrieți pseudocodul buclei unui thread, inclusiv așteptarea la barieră. Realizați o diagramă de timp cu trei threaduri care ajung la barieră în momente diferite.

Termen predare: săptămâna 10

Tema 033.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Executați n faze, începând cu o fază pară. Creați p threaduri și mențineți-le active pe durata sortării. Comparați atribuirea statică în intervale consecutive cu atribuirea ciclică, în care threadul t primește perechile cu indicii t, t+p, t+2p și așa mai departe. Folosiți aceeași barieră între faze în ambele implementări. Pentru două lungimi ale vectorului, comparați cele două atribuiri pentru trei valori ale lui p, folosind aceleași date. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri de calcul.

IS. Realizați o schemă a celor două atribuiri pentru o fază cu opt perechi și trei threaduri. Scrieți pseudocodul parcurgerii perechilor pentru fiecare atribuire. Realizați o diagramă de timp care arată sincronizarea celor trei threaduri la schimbarea fazei.

Termen predare: săptămâna 10

Tema 034.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Executați n faze, începând cu o fază pară. Creați p threaduri și împărțiți perechile fiecărei faze în blocuri consecutive. Fiecare thread preia un indice de bloc folosind Interlocked.Increment pe un contor comun. Reinițializați contorul în acțiunea de după barieră, înainte ca threadurile să înceapă faza următoare. Pentru două lungimi ale vectorului, testați trei dimensiuni ale blocului, cu p constant. Într-o rulare separată, numărați blocurile procesate de fiecare thread în primele două faze. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul de threaduri de calcul.

IS. Realizați o schemă care leagă indicii blocurilor de perechile unei faze. Scrieți pseudocodul preluării blocurilor și al reinițializării contorului. Realizați o diagramă de secvență pentru două threaduri care solicită ultimul bloc disponibil.

Termen predare: săptămâna 10

Tema 035.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Folosiți Parallel.For cu stare locală pentru a detecta dacă o fază a efectuat schimburi. Combinați semnalele locale după procesarea perechilor. Opriți sortarea după două faze

consecutive fără schimburi sau după n faze. Implementați și o variantă care execută întotdeauna n faze cu aceeași distribuire a perechilor. Comparați variantele pentru vectori sortați, aproape sortați și ordonați descrescător, de aceeași lungime. Păstrați același MaxDegreeOfParallelism și raportați numărul de faze executate. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă a reducerii semnalelor locale. Scrieți pseudocodul actualizării numărului de faze consecutive fără schimburi. Realizați o diagramă de stări a criteriului de oprire.

Termen predare: săptămâna 10

Tema 036.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Executați n faze, începând cu o fază pară. Folosiți Parallel.ForEach și Partitioner.Create pentru a grupa indicii perechilor în intervale consecutive. O iterație procesează secvențial toate perechile din intervalul primit. Construiți intervalele separat pentru fazele pare și impare și așteptați terminarea fiecărei bucle înainte de a avansa. Pentru două lungimi ale vectorului, testați trei dimensiuni de interval la același MaxDegreeOfParallelism. Comparați și cu o implementare Parallel.For cu o pereche per iterație. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă de grupare a perechilor, inclusiv un ultim interval incomplet. Scrieți pseudocodul procesării unui interval. Realizați o diagramă de secvență care arată cum sunt preluate intervalele unei faze și când începe faza următoare.

Termen predare: săptămâna 10

Tema 037.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Executați n faze, începând cu o fază pară. Împărțiți vectorul în blocuri consecutive de lungime pară, cu posibilitatea ca ultimul bloc să fie mai scurt. În fiecare fază, calculați cu Parallel.For perechile din interiorul blocurilor. Apoi procesați secvențial perechile care traversează limitele blocurilor, dacă există. Terminați ambele etape înainte de a trece la faza următoare. Pentru două lungimi ale vectorului, testați trei dimensiuni pare ale blocului la același MaxDegreeOfParallelism. Raportați numărul perechilor de la limitele blocurilor într-o fază pară și într-una impară. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă cu perechile interne și cele de la limite pentru două faze. Scrieți pseudocodul identificării perechilor de la limite. Realizați o diagramă de timp cu etapa paralelă, etapa secvențială și trecerea la faza următoare.

Termen predare: săptămâna 10

Tema 038.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Executați n faze, începând cu o fază pară. Folosiți doi vectori. În fiecare fază, citiți valorile din vectorul sursă și scrieți în vectorul destinație minimul și maximul fiecărei perechi, folosind Parallel.For. Copiați și elementele care nu fac parte dintr-o pereche. După terminarea fazei, inversați rolurile vectorilor. Pentru două lungimi ale vectorului, comparați varianta cu doi vectori cu o variantă Parallel.For care face schimburile direct în vector. Păstrați același MaxDegreeOfParallelism și includeți alocarea vectorului auxiliar în timpul măsurat. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Realizați o schemă a citirilor și scrierilor într-o fază cu elemente fără pereche. Scrieți pseudocodul copierii acestor elemente și al schimbării rolurilor vectorilor. Realizați o diagramă de timp care arată de ce sursa rămâne disponibilă până la sfârșitul fazei.

Termen predare: săptămâna 10

Tema 039.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Executați n faze, începând cu o fază pară. La fiecare fază, un thread producător adaugă în BlockingCollection intervale de indici ai perechilor. Folosiți o coadă cu capacitate limitată. Threadurile consumatoare procesează intervalele până la închiderea și golirea cozii. Începeți faza următoare după terminarea tuturor threadurilor din faza curentă. Alegeți două lungimi pentru care execuția rămâne fezabilă. Testați trei capacități ale cozii la același număr de consumatori și aceeași dimensiune a intervalului. Includeți pornirea threadurilor și transferul intervalelor prin coadă în timpul sortării. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp. Calculați eficiența E = S / p și costul paralel C = p × Tp, unde p este numărul total de threaduri producătoare și consumatoare dintr-o fază.

IS. Realizați o schemă de flux al datelor de la producător la perechile din vector. Scrieți pseudocodul unei faze, cu semnalarea încheierii adăugării. Realizați o diagramă de stări a consumatorului, cu așteptarea, procesarea și terminarea.

Termen predare: săptămâna 10

Tema 040.

APD1. Implementați în C# pe CPU sortarea Odd–Even a unui vector de n numere întregi. În fazele pare, procesați perechile (0, 1), (2, 3) și așa mai departe. În fazele impare, procesați perechile (1, 2), (3, 4) și așa mai departe. Pentru fiecare pereche, ordonați crescător cele două valori. Executați n faze, începând cu o fază pară. Construiți câte un task pentru fiecare bloc de perechi dintr-o fază, fără a impune o barieră globală între faze. Pentru fiecare task, identificați taskurile din faza precedentă care au scris în pozițiile pe care acesta le citește sau le actualizează. Porniți-l după terminarea acestor dependențe, folosind Task.WhenAll și await. Includeți în dependențe și ultimul task care a accesat fiecare poziție, dacă poziția nu a fost procesată în faza precedentă. Așteptați toate taskurile înainte de a citi vectorul sortat. Pentru două lungimi fezabile, comparați trei dimensiuni ale blocului de perechi cu o variantă care așteaptă toate taskurile la fiecare fază. Păstrați aceleași blocuri în cele două variante. Includeți construirea dependențelor în timpul măsurat. Verificați rezultatul față de Odd–Even Sort secvențial, inclusiv pentru un vector cu duplicate. Măsurați timpul sortării complete, cu datele deja în memorie. Pregătiți copii identice ale vectorului inițial înainte de cronometrare. După o rulare de încălzire pentru fiecare configurație, efectuați cel puțin cinci măsurări și raportați media aritmetică a timpilor. Notați cu Ts timpul secvențial și cu Tp timpul paralel. Calculați accelerarea S = Ts / Tp.

IS. Desenați graful dependențelor pentru primele trei faze ale unui vector cu șapte elemente. Scrieți pseudocodul construirii dependențelor folosind ultimul task asociat fiecărei poziții. Realizați o diagramă de timp care arată când un task poate porni înainte de terminarea tuturor taskurilor fazei precedente.

Termen predare: săptămâna 10